09

Implementação com Multi-Agents

Trabalhar com um único agent é pair programming: síncrono, sequencial, limitado a uma janela de contexto. Multi-agents é outra profissão — orquestração de equipe. E como toda mudança de paradigma, exige habilidades novas: decompor trabalho com clareza, verificar outputs por contrato e saber quando confiar. Escrever código deixa de ser a habilidade central.

A IDEIA CENTRAL DESTE MATERIAL

Todo dev bate em três limites com um agent só: sobrecarga de contexto, falta de especialização e ausência de coordenação. Multi-agents resolve os dois primeiros de graça — o terceiro é resolvido pelo PLAN.md e pelas regras deste material.

1 Os três modos de execução

MODO	COMO FUNCIONA	QUANDO USAR
1 — Sequencial por fase	Um agent executa as tasks da fase em sequência; você valida cada output antes do próximo.	Para começar, para tasks com dependências fortes, para aprender o fluxo. Mais controle, menos velocidade.
2 — Paralelo dentro da fase	Múltiplos agents simultâneos, cada um com uma task independente da mesma fase — cada um com janela de contexto isolada.	Quando o PLAN.md já mapeou a independência (material 07).
3 — Orquestrador + sub-agents	Um agent principal lê o PLAN.md e delega: especialistas pegam tasks do seu domínio, marcam "em progresso" (evitando conflito) e sinalizam outputs ao concluir.	Projetos maiores, depois que os modos 1 e 2 estão dominados.

2 Sub-agents vs Agent Teams

A pergunta decisiva: **os workers precisam conversar entre si?**

SUB-AGENTS — WORKERS QUE REPORTAM

Tasks paralelas independentes, análise de módulos separados, features isoladas.

Mais leves, mais baratos. Reportam ao agent principal e pronto.

AGENT TEAMS — COLEGAS QUE SE COORDENAM

Coordenação cross-layer (frontend + backend + testes), debugging com hipóteses concorrentes.

Compartilham descobertas e se desafiam — ao custo de **muito mais tokens** e overhead de coordenação.

> Para tasks sequenciais, edições no mesmo arquivo ou trabalho cheio de dependências: **uma sessão única ou sub-agents** são mais eficientes que um time. Time é para quando a independência é real.

3 Claude Code: os modos de multi-agent

Sub-agents via Task tool

O Claude Code não é "um AI": é um **sistema de orquestração**. A sessão principal é o cérebro central que delega para agents especialistas via Task tool — em modo paralelo, sequencial ou background. A qualidade do resultado depende de quão bem você ensinou esse cérebro central a delegar.

Agents customizados por domínio

Especialistas são arquivos Markdown com frontmatter YAML em `.claude/agents/`:

```
---
name: backend-agent
description: Implementa endpoints, modelos e lógica de serviço
tools: Read, Write, Edit, Bash
---

Você é um engineer especializado em backend.
Siga as convenções definidas no CLAUDE.md.
Escreva testes antes de implementar (TDD).
```

Agents do projeto (`.claude/agents/`) são versionados e compartilháveis com o time; agents do usuário (`~/.claude/agents/`) valem em todos os projetos. Ambos herdam o contexto do CLAUDE.md automaticamente.

Agent Teams

Recurso mais recente (experimental à época deste material — verifique o estado atual na sua versão): uma sessão vira "team lead" coordenando teammates via lista de tasks compartilhada. Cada teammate roda em janela própria e eles se comunicam **diretamente entre si** — a diferença fundamental para sub-agents, que só reportam ao principal.

4 OpenCode: como o sistema de agents funciona

TIPO	AGENTS	INVOCAÇÃO
Primários	`build` (dev completo) e `plan` (análise, edições negadas)	Alternados com **Tab** — nunca com @
Sub-agents nativos	`@general` (acesso total, tasks multi-step) e `@explore` (somente leitura, busca no codebase)	**@mention** — cria contexto isolado

```
# Sub-agent com acesso total implementando uma task
@general Leia o PLAN.md e execute a Task 2.1 completa,
         com testes primeiro (TDD), seguindo o AGENTS.md

# Sub-agent somente leitura explorando
@explore Encontre todos os arquivos do módulo de autenticação

# Paralelo real: dois terminais, duas sessões independentes
Terminal 1: @general Execute a Task 2.1 do PLAN.md com TDD
Terminal 2: @general Execute a Task 2.2 do PLAN.md com TDD
```

Agents customizados vão em `.opencode/agents/nome.md` com `mode: subagent` no frontmatter — após reiniciar, o `@nome` aparece no autocomplete. Limitação conhecida à época: agents customizados são invocáveis por **você** via @, mas o Task tool interno só delega para `explore` e `general` programaticamente. O OpenCode também vem implementando Agent Teams com um diferencial: **múltiplos providers no mesmo time** (Claude orquestrando, outros modelos executando).

5 Padrões de orquestração

PADRÃO	COMO FUNCIONA	O QUE CAPTURA
Builder-Validator	Um agent constrói, outro valida antes de qualquer merge (o reviewer do material 10).	Incompatibilidades de interface e suposições quebradas — antes de chegarem à produção.
Split-and-Merge	O orquestrador divide uma análise grande; agents paralelos analisam partes sem saber uns dos outros; o orquestrador sintetiza.	Análises que não cabem numa janela de contexto.
Waves	Tasks agrupadas em ondas por dependência; onda inteira roda em paralelo; a próxima só começa com os testes da anterior passando.	É exatamente o que as fases do PLAN.md definem.

6 As quatro regras anti-conflito

1. **Uma task, um agent.** Nunca a mesma task para dois agents — o PLAN.md define a atribuição, e agents marcam a task "em progresso" antes de começar.
2. **Nada de mesmo arquivo em paralelo.** Duas tasks tocam o mesmo arquivo? São sequenciais. Reorganize o plano.
3. **Contexto isolado por agent.** Cada sub-agent nasce com contexto limpo, sem as suposições acumuladas da sessão principal — é isso que dá "olhos frescos" ao review.
4. **Gate de fase inegociável.** Nenhum agent da próxima fase começa antes do critério da fase atual passar. É o que impede o paralelismo de virar integração quebrada.

7 Custo: tokens e modelo por agent

Um padrão eficiente: sessão principal num modelo mais forte para o raciocínio de orquestração; sub-agents num modelo mais leve para tasks bem definidas. No Claude Code:

```
export CLAUDE_CODE_SUBAGENT_MODEL="claude-sonnet-4-6"
```

> **Aritmética honesta:** três agents paralelos queimam seus rate limits ~3× mais rápido que um agent fazendo o mesmo trabalho em sequência. Paralelismo compra *tempo*, não *tokens*. Verifique os limites do seu plano antes de montar times grandes.

8 Prompt pronto: executando uma fase

▸ PROMPT PARA INICIAR A IMPLEMENTAÇÃO

```
Leia SPEC.md, CLAUDE.md (ou AGENTS.md) e PLAN.md na raiz.
O scaffold está completo e os smoke tests passam.

Execute a [FASE N] do PLAN.md seguindo estas regras:

1. ANTES DE COMEÇAR
   Identifique quais tasks da fase são genuinamente independentes
   (paralelo) e quais têm dependências (sequência).

2. PARA CADA TASK
   - Leia Input, Output e Testes críticos
   - Escreva os testes PRIMEIRO (red)
   - Confirme que falham antes de implementar
   - Implemente o mínimo para os testes passarem (green)
   - Marque o checkbox da task no PLAN.md

3. GATE DA FASE
   Ao concluir todas as tasks, execute o critério de conclusão
   da fase. Só avance após o gate passar.

4. RESTRIÇÕES
   - Siga as convenções do CLAUDE.md/AGENTS.md
   - Não implemente nada fora do PLAN.md
   - Task bloqueada? Sinalize e aguarde — não invente
   - Não edite SPEC.md, CLAUDE.md ou PLAN.md (além dos checkboxes)

Ao terminar: liste tasks concluídas, testes passando
e o que a próxima fase precisa para começar.
```

▸ **COLA PARA FIXAR**

- Um agent = pair programming; multi-agents = **orquestração de equipe** — e decompor bem vira a habilidade central.
- Três modos: sequencial (controle), paralelo na fase (velocidade), orquestrador + sub-agents (escala).
- Sub-agents **reportam**; Agent Teams **conversam** — e times custam caro. Use a pergunta: os workers precisam se comunicar?
- Padrões que funcionam: **Builder-Validator**, **Split-and-Merge**, **Waves** (= fases do PLAN.md).
- Anti-conflito: uma task um agent, nunca o mesmo arquivo em paralelo, contexto isolado, gate de fase sagrado.
- Paralelismo compra tempo, não tokens: **3 agents ≈ 3× o consumo**.